

SkillForge AI

Sample Lesson Content · Quality Validation Pack

5 lessons across Java, Python, React, System Design & AWS

AI-Generated Lesson Content — Upskilling Platform Sample

This document demonstrates the depth and quality of AI-generated lesson content produced by the platform. Each lesson below is a full, production-quality educational unit.

Java Mastery — Introduction & Core Concepts

Type: Reading | **Duration:** 20 min | **XP:** 50

The Java Virtual Machine: Architecture & Bytecode

Java's defining promise — "Write Once, Run Anywhere" — is delivered by the JVM. When you compile Java source code, `javac` does not produce native machine code. Instead, it

produces **bytecode**: a compact, platform-neutral instruction set stored in `.class` files. The JVM is a software runtime that interprets (or JIT-compiles) this bytecode into native instructions specific to the host OS and CPU.

The JVM has three primary subsystems:

- **Class Loader Subsystem** — Loads, links, and initialises `.class` files at runtime.
- **Runtime Data Areas** — Memory regions: Heap, Stack, Method Area (Metaspace in Java 8+), PC Registers, Native Method Stacks.
- **Execution Engine** — Interpreter, JIT Compiler, and Garbage Collector.

Class Loading in Detail

Class loading happens in three phases:

1. **Loading** — The bootstrap, extension, or application class loader reads the `.class` file bytes.
2. **Linking** — Bytecode verification (is it structurally valid?), preparation (static fields set to defaults), and resolution (symbolic references replaced with direct references).
3. **Initialisation** — Static initialisers and static field assignments run for the first time.

Pro Tip: Class loading is lazy by default. A class is loaded the first time it is actively used (e.g., instantiation, static method call). You can force loading via `Class.forName("com.example.MyClass")`, but initialisation only happens if the second argument is `true`.

Java Memory Model: Heap, Stack & Metaspace

Area	Stores	GC-managed?
Heap	Object instances, arrays	Yes
Stack	Method frames, local primitives, object references	No (LIFO pop)
Metaspace	Class metadata, method bytecode, static variables (Java 8+)	Partially
	Address of current instruction per thread	No

Area	Stores	GC-managed?
PC Register		

- Each **thread** gets its own Stack and PC Register.
- The **Heap** is shared across all threads — hence the need for synchronisation.
- **Metaspace** replaced PermGen in Java 8 and grows dynamically by default (set `-XX:MaxMetaspaceSize` to cap it).

▲ **Warning:** A `StackOverflowError` means your call stack is exhausted — usually from runaway recursion. An `OutOfMemoryError: Java heap space` means heap is full. An `OutOfMemoryError: Metaspace` means class metadata exceeds limits — common in frameworks that generate many proxy classes (e.g., Spring with many beans).

The Four Pillars of OOP in Java

1. Encapsulation

Bundle state (fields) and behaviour (methods) together; hide internal state behind access modifiers (`private` , `protected` , `public`).

2. Inheritance

A subclass (`extends`) inherits fields and methods from a superclass, enabling code reuse. Java supports single-class inheritance but multiple interface inheritance.

3. Polymorphism

The same interface can refer to different concrete types. Method overriding at runtime (dynamic dispatch) and method overloading at compile time.

4. Abstraction

Hide implementation complexity behind abstract classes or interfaces. The caller knows what to do, not how it's done.

Modern Java Features: var, Records, and Sealed Classes

Local Variable Type Inference (`var`) — Java 10+

```
// Before var
Map<String, List<Integer>> scores = new HashMap<String, List<Integer>>();

// With var - type is inferred, still statically typed
var scores = new HashMap<String, List<Integer>>();
```

`var` only works for local variables with an initialiser. It is not `dynamic` typing — the type is fixed at compile time.

Records — Java 16+

Records are immutable data carriers. The compiler auto-generates constructor, accessors, `equals()`, `hashCode()`, and `toString()`.

```
public record Point(double x, double y) {
    // Compact canonical constructor for validation
    public Point {
        if (Double.isNaN(x) || Double.isNaN(y)) {
            throw new IllegalArgumentException("Coordinates must be valid numbers");
        }
    }

    public double distanceTo(Point other) {
        double dx = this.x - other.x;
        double dy = this.y - other.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}

// Usage
var origin = new Point(0, 0);
var target = new Point(3, 4);
System.out.println(origin.distanceTo(target)); // 5.0
```

Sealed Classes — Java 17+

Sealed classes restrict which classes can extend or implement them. This enables exhaustive pattern matching.

```

public sealed interface Shape permits Circle, Rectangle, Triangle {}

public record Circle(double radius) implements Shape {}
public record Rectangle(double width, double height) implements Shape {}
public record Triangle(double base, double height) implements Shape {}

// Exhaustive switch (Java 21 pattern matching)
double area = switch (shape) {
    case Circle c    → Math.PI * c.radius() * c.radius();
    case Rectangle r → r.width() * r.height();
    case Triangle t  → 0.5 * t.base() * t.height();
};

```

Full OOP Example: All Four Pillars Together

```

import java.util.ArrayList;
import java.util.List;

// ABSTRACTION: abstract class defines contract, hides implementation
public abstract class BankAccount {
    private final String accountNumber; // ENCAPSULATION: private field
    private double balance;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    // Encapsulation: controlled access via getter
    public String getAccountNumber() { return accountNumber; }
    public double getBalance() { return balance; }

    protected void setBalance(double balance) { this.balance = balance; }

    // Abstraction: subclasses define deposit rules
    public abstract void deposit(double amount);

    // Concrete shared behaviour
    public void withdraw(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("Amount must be positive");
        if (amount > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amount;
        logTransaction("WITHDRAWAL", amount);
    }

    protected void logTransaction(String type, double amount) {

```

```

        System.out.printf("[%s] %s: %.2f | Balance: %.2f%n",
            accountNumber, type, amount, balance);
    }

    @Override
    public String toString() {
        return "%s[account=%s, balance=%.2f]"
            .formatted(getClass().getSimpleName(), accountNumber, balance);
    }
}

// INHERITANCE: SavingsAccount extends BankAccount
public class SavingsAccount extends BankAccount {
    private final double interestRate;

    public SavingsAccount(String accountNumber, double initialBalance, double interestRate) {
        super(accountNumber, initialBalance);
        this.interestRate = interestRate;
    }

    @Override // POLYMORPHISM: overrides abstract method
    public void deposit(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("Amount must be positive");
        setBalance(getBalance() + amount);
        logTransaction("DEPOSIT", amount);
    }

    public void applyInterest() {
        double interest = getBalance() * interestRate;
        deposit(interest);
        System.out.println("Interest applied: " + interest);
    }
}

// INHERITANCE: CheckingAccount with overdraft feature
public class CheckingAccount extends BankAccount {
    private final double overdraftLimit;

    public CheckingAccount(String accountNumber, double initialBalance, double overdraftLimit) {
        super(accountNumber, initialBalance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public void deposit(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("Amount must be positive");
        setBalance(getBalance() + amount);
        logTransaction("DEPOSIT", amount);
    }

    @Override // POLYMORPHISM: overrides concrete method too
    public void withdraw(double amount) {
        if (amount ≤ 0) throw new IllegalArgumentException("Amount must be positive");

```

```

        if (amount > getBalance() + overdraftLimit) {
            throw new IllegalStateException("Exceeds overdraft limit");
        }
        setBalance(getBalance() - amount);
        logTransaction("WITHDRAWAL", amount);
    }
}

// POLYMORPHISM in action: same type reference, different behaviour
public class Bank {
    public static void main(String[] args) {
        List<BankAccount> accounts = new ArrayList<>();
        accounts.add(new SavingsAccount("SAV-001", 1000.0, 0.05));
        accounts.add(new CheckingAccount("CHK-001", 500.0, 200.0));

        for (BankAccount account : accounts) {
            account.deposit(250.0); // Each calls its own overridden method
            System.out.println(account);
        }
    }
}

```

Key Points

- The JVM compiles bytecode to native code at runtime via the JIT compiler — hotspot methods get compiled to fast machine code after repeated execution.
 - Stack memory is thread-local and auto-managed; heap memory is shared and GC-managed.
 - `var` is syntactic sugar — the type is fully known at compile time, just inferred.
 - Records are `final` by default and cannot extend other classes (but can implement interfaces).
 - Sealed classes + pattern matching switches give you the power of algebraic data types from functional languages, with compile-time exhaustiveness checking.
-

Quiz

Q1. Which JVM memory area stores class metadata and static variables in Java 8+? - A) Heap - B) Stack - C) Metaspace ✓ - D) PermGen

Explanation: PermGen was replaced by Metaspace in Java 8. Metaspace stores class metadata and grows dynamically.

Q2. What does the `sealed` keyword enforce? - A) That a class cannot be instantiated - B) That only explicitly listed classes can extend or implement it ✓ - C) That all fields must be private - D) That the class is thread-safe

Q3. In the OOP example above, when `account.deposit(250.0)` is called on a `BankAccount` reference holding a `SavingsAccount` object, which method runs? - A) `BankAccount.deposit()` — because the reference type is `BankAccount` - B) `SavingsAccount.deposit()` — because Java uses dynamic dispatch ✓ - C) Both methods run in sequence - D) A compile error occurs since `BankAccount.deposit()` is abstract

Python for Everyone — Deep Dive & Advanced Patterns

Type: Reading | **Duration:** 35 min | **XP:** 75

Comprehensions: Concise, Readable Data Transformations

Comprehensions are one of Python's most expressive features. They replace verbose `for` loops with a single, readable expression.

List Comprehensions

```
# Traditional approach
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x ** 2)

# Comprehension: [expression for item in iterable if condition]
squares = [x ** 2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]

# Nested comprehension: flatten a matrix
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
flat = [cell for row in matrix for cell in row]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Dict and Set Comprehensions

```
# Dict comprehension: word → length mapping
words = ["python", "java", "rust", "go"]
word_lengths = {word: len(word) for word in words}
# {'python': 6, 'java': 4, 'rust': 4, 'go': 2}

# Invert a dict (only works if values are unique)
inverted = {v: k for k, v in word_lengths.items()}

# Set comprehension: unique vowels in a string
sentence = "the quick brown fox"
vowels = {ch for ch in sentence if ch in "aeiou"}
# {'e', 'i', 'o', 'u'}
```

Generator Expressions: Lazy Evaluation

Generators look like list comprehensions but use `()`. They produce values one at a time — no memory overhead for large sequences.

```
import sys

# List: all values stored in memory immediately
list_sq = [x ** 2 for x in range(1_000_000)]
print(sys.getsizeof(list_sq))    # ~8 MB

# Generator: computes lazily, one value per next()
gen_sq = (x ** 2 for x in range(1_000_000))
print(sys.getsizeof(gen_sq))    # ~112 bytes

# Generators compose efficiently
total = sum(x ** 2 for x in range(1_000_000) if x % 3 == 0)
```

Pro Tip: Prefer generator expressions over list comprehensions when you only iterate once and don't need random access. Use `list()` to materialise when you need to iterate multiple times or need indexing.

Decorators: Functions That Transform Functions

A decorator is a function that takes a function and returns a new function. They implement the **Decorator design pattern** natively in Python syntax.

```
import functools
import time
import logging

# Basic decorator: logs function calls
def log_calls(func):
    @functools.wraps(func) # Preserves func's __name__, __doc__, etc.
    def wrapper(*args, **kwargs):
        logging.info(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")
        result = func(*args, **kwargs)
        logging.info(f"{func.__name__} returned {result}")
        return result
    return wrapper

# Decorator factory: decorator that accepts arguments
def retry(max_attempts: int = 3, delay: float = 1.0, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_error = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_error = e
                    if attempt < max_attempts:
                        print(f"Attempt {attempt} failed: {e}. Retrying in {delay}s...")
                        time.sleep(delay)
            raise last_error
        return wrapper
    return decorator

# Combining decorators – applied bottom-up
@log_calls
@retry(max_attempts=3, delay=0.5, exceptions=(ConnectionError,))
def fetch_data(url: str) → dict:
    # Simulates an unreliable network call
    import random
    if random.random() < 0.5:
        raise ConnectionError("Network timeout")
    return {"status": "ok", "url": url}
```

⚠ **Warning:** Always use `@functools.wraps(func)` inside your wrapper. Without it, the wrapper replaces `func.__name__` and `func.__doc__` with wrapper's, which breaks introspection tools, docstring generators, and debuggers.

Class-Based Decorators

```
class memoize:
    """Decorator that caches return values."""
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func = func
        self.cache = {}

    def __call__(self, *args):
        if args not in self.cache:
            self.cache[args] = self.func(*args)
        return self.cache[args]

@memoize
def fibonacci(n: int) → int:
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(50)) # Instant; without memoize this would take ages
```

Context Managers: Guaranteed Cleanup

Context managers implement the `__enter__` / `__exit__` protocol to ensure resources are always cleaned up, even when exceptions occur.

```
import sqlite3
from contextlib import contextmanager

# Class-based context manager
class DatabaseTransaction:
    def __init__(self, db_path: str):
        self.db_path = db_path
        self.conn = None
        self.cursor = None

    def __enter__(self):
```

```

        self.conn = sqlite3.connect(self.db_path)
        self.cursor = self.conn.cursor()
        return self.cursor # Value bound to 'as' target

def __exit__(self, exc_type, exc_val, exc_tb):
    if exc_type is None:
        self.conn.commit() # Success: commit
        print("Transaction committed")
    else:
        self.conn.rollback() # Exception: rollback
        print(f"Transaction rolled back due to: {exc_val}")
    self.cursor.close()
    self.conn.close()
    return False # Don't suppress the exception

# Usage
with DatabaseTransaction("app.db") as cursor:
    cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT)")
    cursor.execute("INSERT INTO users (name) VALUES (?)", ("Alice",))
    # If anything raises here, rollback happens automatically

# Generator-based context manager using @contextmanager
@contextmanager
def temp_directory():
    import tempfile, shutil
    tmpdir = tempfile.mkdtemp()
    try:
        print(f"Created temp dir: {tmpdir}")
        yield tmpdir # Execution pauses here for the with block
    finally:
        shutil.rmtree(tmpdir) # Always runs, even on exception
        print(f"Cleaned up: {tmpdir}")

with temp_directory() as d:
    # Work with d safely; cleanup is guaranteed
    with open(f"{d}/test.txt", "w") as f:
        f.write("temporary data")

```

Dataclasses: Structured Data Without Boilerplate

```

from dataclasses import dataclass, field, asdict
from typing import Optional
import json

@dataclass(order=True, frozen=False)
class Employee:
    # sort_index is used for ordering, excluded from __init__ display

```

```

    sort_index: float = field(init=False, repr=False)
    employee_id: str
    name: str
    department: str
    salary: float
    skills: list[str] = field(default_factory=list)
    manager: Optional["Employee"] = field(default=None, repr=False)

    def __post_init__(self):
        self.sort_index = self.salary # Order by salary

    def give_raise(self, percentage: float) → None:
        if percentage ≤ 0:
            raise ValueError("Raise percentage must be positive")
        self.salary *= (1 + percentage / 100)
        self.sort_index = self.salary

    def to_json(self) → str:
        d = asdict(self)
        d.pop("sort_index", None)
        return json.dumps(d, indent=2)

# Usage
alice = Employee("E001", "Alice Chen", "Engineering", 120_000, ["Python", "AWS"])
bob   = Employee("E002", "Bob Torres", "Engineering", 95_000, ["Java", "K8s"], manager=alice)

alice.give_raise(10)
print(alice.salary)    # 132000.0
print(alice > bob)     # True (ordered by salary)
print(alice.to_json())

```

Type Hints with Generics

```

from typing import TypeVar, Generic, Iterator
from collections.abc import Callable

T = TypeVar("T")
U = TypeVar("U")

class Pipeline(Generic[T]):
    """A composable data processing pipeline."""

    def __init__(self, data: list[T]):
        self._data = data

    def map(self, func: Callable[[T], U]) → "Pipeline[U]":
        return Pipeline([func(item) for item in self._data])

```

```

def filter(self, predicate: Callable[[T], bool]) → "Pipeline[T]":
    return Pipeline([item for item in self._data if predicate(item)])

def reduce(self, func: Callable[[U, T], U], initial: U) → U:
    result = initial
    for item in self._data:
        result = func(result, item)
    return result

def __iter__(self) → Iterator[T]:
    return iter(self._data)

# Fully type-safe pipeline
result = (
    Pipeline(range(1, 21))
    .filter(lambda x: x % 2 == 0)
    .map(lambda x: x ** 2)
    .reduce(lambda acc, x: acc + x, 0)
)
print(result) # Sum of squares of even numbers 1-20 = 1540

```

itertools & functools Patterns

```

import itertools
import functools
from collections import Counter

# itertools.groupby: group sorted data
employees = [
    ("Engineering", "Alice"), ("Engineering", "Bob"),
    ("Marketing", "Carol"), ("Engineering", "Dave"),
    ("Marketing", "Eve"),
]
employees.sort(key=lambda e: e[0]) # Must sort before groupby!
for dept, members in itertools.groupby(employees, key=lambda e: e[0]):
    print(f"{dept}: {[m[1] for m in members]}")

# itertools.chain: flatten iterables without copying
lists = [[1, 2], [3, 4], [5, 6]]
flat = list(itertools.chain.from_iterable(lists)) # [1, 2, 3, 4, 5, 6]

# itertools.islice: take first N from a generator
first_five_even_squares = list(
    itertools.islice((x**2 for x in itertools.count(1) if x % 2 == 0), 5)
)
# [4, 16, 36, 64, 100]

```

```
# functools.partial: pre-fill arguments
def power(base, exponent):
    return base ** exponent

square = functools.partial(power, exponent=2)
cube   = functools.partial(power, exponent=3)
print(list(map(square, [1, 2, 3, 4, 5]))) # [1, 4, 9, 16, 25]

# functools.reduce with operator
import operator
factorial = functools.reduce(operator.mul, range(1, 11), 1)
print(factorial) # 3628800
```

Key Points

- List comprehensions are faster than equivalent `for` loops + `append()` because they're optimised at the C level in CPython.
- Generator expressions are memory-efficient; use them for pipelines where each element is consumed once.
- `@functools.wraps` is not optional — it's essential for debugging, documentation, and frameworks that inspect function metadata.
- Context managers guarantee cleanup via `__exit__` even if an exception propagates; `return False` from `__exit__` re-raises the exception, `return True` suppresses it.
- Dataclasses with `frozen=True` produce hashable, immutable objects suitable as dict keys or set members.
- Type hints are not enforced at runtime by Python itself — use `mypy` or `pyright` for static analysis.

Quiz

Q1. What is the memory advantage of `(x**2 for x in range(10**6))` over `[x**2 for x in range(10**6)]` ? - A) The generator is faster to compute - B) The generator stores only one value at a time, using constant memory ✓ - C) There is no difference - D) Generators are automatically parallelised

- Q2.** What happens if you omit `@functools.wraps(func)` in a decorator? - A) The decorator will not work at all - B) The wrapped function loses its original `__name__`, `__doc__`, and signature ✓ - C) The decorator is applied twice - D) An ImportError is raised
- Q3.** In a context manager's `__exit__` method, what does returning `True` do? - A) Commits any database transaction - B) Suppresses the exception that triggered the exit ✓ - C) Forces the `with` block to re-run - D) Closes the file handle
-

React 19 & Ecosystem — Build a GitHub User Search Component

Type: Exercise | **Duration:** 50 min | **XP:** 100

Overview

In this exercise you will build a **real-time GitHub user search** feature. By the end you will have:

- A debounced search input that delays API calls until the user stops typing
- A custom `useDebounce` hook that is reusable across the app
- Proper loading and error states with user-friendly UI
- A memoised result renderer that avoids unnecessary re-renders
- Full TypeScript types throughout

Prerequisites: Node 20+, a React 19 project (created with `npm create vite@latest my-app -- --template react-ts`).

Step 1: Create the `useDebounce` Custom Hook

Create `/src/hooks/useDebounce.ts` :

```
import { useState, useEffect } from "react";

/**
 * Delays updating the returned value until `delay` ms have elapsed
 * since the last change to `value`.
 */
export function useDebounce<T>(value: T, delay: number = 400): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    // Set a timer to update the debounced value after the delay
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup: if value changes before delay elapses, cancel the previous timer
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

Pro Tip: The cleanup function returned from `useEffect` runs before the next effect invocation and on unmount. This is what makes debouncing work: every keystroke cancels the previous timeout and starts a fresh one. Only when the user pauses does the timeout complete and the debounced value update.

Step 2: Define TypeScript Types

Create `/src/types/github.ts` :

```
export interface GitHubUser {
  login: string;
  id: number;
  avatar_url: string;
  html_url: string;
  type: "User" | "Organization";
  score: number;
}

export interface GitHubSearchResponse {
  total_count: number;
  incomplete_results: boolean;
}
```

```

    items: GitHubUser[];
  }

  export type SearchStatus = "idle" | "loading" | "success" | "error";

```

Step 3: Create the Search Service

Create `/src/services/githubApi.ts`:

```

import type { GitHubSearchResponse } from "../types/github";

const BASE_URL = "https://api.github.com";

export class GitHubApiError extends Error {
  constructor(
    public status: number,
    message: string
  ) {
    super(message);
    this.name = "GitHubApiError";
  }
}

export async function searchUsers(
  query: string,
  signal?: AbortSignal
): Promise<GitHubSearchResponse> {
  if (!query.trim()) {
    return { total_count: 0, incomplete_results: false, items: [] };
  }

  const url = `${BASE_URL}/search/users?q=${encodeURIComponent(query)}&per_page=10`;
  const response = await fetch(url, {
    signal,
    headers: {
      Accept: "application/vnd.github.v3+json",
      // Add: Authorization: `Bearer ${token}` for higher rate limits
    },
  });

  if (!response.ok) {
    if (response.status === 403) {
      throw new GitHubApiError(403, "Rate limit exceeded. Try again in a minute.");
    }
    throw new GitHubApiError(response.status, `GitHub API error: ${response.statusText}`);
  }
}

```

```
    return response.json() as Promise<GitHubSearchResponse>;  
  }  
}
```

Step 4: Build the Main Search Hook

Create `/src/hooks/useGitHubSearch.ts` :

```
import { useState, useEffect, useCallback, useRef } from "react";  
import type { GitHubUser, SearchStatus } from "../types/github";  
import { searchUsers, GitHubApiError } from "../services/githubApi";  
import { useDebounce } from "../useDebounce";  
  
interface UseGitHubSearchResult {  
  query: string;  
  setQuery: (query: string) => void;  
  users: GitHubUser[];  
  totalCount: number;  
  status: SearchStatus;  
  errorMessage: string | null;  
  clearResults: () => void;  
}  
  
export function useGitHubSearch(): UseGitHubSearchResult {  
  const [query, setQuery] = useState("");  
  const [users, setUsers] = useState<GitHubUser[]>([]);  
  const [totalCount, setTotalCount] = useState(0);  
  const [status, setStatus] = useState<SearchStatus>("idle");  
  const [errorMessage, setErrorMessage] = useState<string | null>(null);  
  
  const debouncedQuery = useDebounce(query, 450);  
  const abortControllerRef = useRef<AbortController | null>(null);  
  
  const clearResults = useCallback(() => {  
    setQuery("");  
    setUsers([]);  
    setTotalCount(0);  
    setStatus("idle");  
    setErrorMessage(null);  
  }, []);  
  
  useEffect(() => {  
    // Cancel any in-flight request  
    abortControllerRef.current?.abort();  
  
    if (!debouncedQuery.trim()) {  
      setUsers([]);  
      setTotalCount(0);  
    }  
  }, [debouncedQuery]);  
  
  useEffect(() => {  
    if (!debouncedQuery.trim()) return;  
    const controller = new AbortController();  
    abortControllerRef.current = controller;  
    searchUsers(debouncedQuery, controller.signal).then(  
      (response) => {  
        setUsers(response.users);  
        setTotalCount(response.totalCount);  
        setStatus(response.status);  
        if (response.errorMessage) {  
          setErrorMessage(response.errorMessage);  
        }  
      })  
    );  
  }, [debouncedQuery]);  
}
```

```

        setStatus("idle");
        return;
    }

    const controller = new AbortController();
    abortControllerRef.current = controller;

    setStatus("loading");
    setErrorMessage(null);

    searchUsers(debouncedQuery, controller.signal)
        .then((data) => {
            if (!controller.signal.aborted) {
                setUsers(data.items);
                setTotalCount(data.total_count);
                setStatus("success");
            }
        })
        .catch((error) => {
            if (error.name === "AbortError") return; // Intentional cancellation

            setStatus("error");
            if (error instanceof GitHubApiError) {
                setErrorMessage(error.message);
            } else {
                setErrorMessage("An unexpected error occurred. Check your connection.");
            }
        });

    return () => controller.abort();
}, [debouncedQuery]);

return { query, setQuery, users, totalCount, status, errorMessage, clearResults };
}

```

⚠ Warning: Always abort fetch requests when a component unmounts or the query changes. Without aborting, a slow response from an old query can arrive after a newer one and overwrite the correct results — a classic race condition. The `AbortController` pattern above prevents this entirely.

Step 5: Build the UI Components

Create `/src/components/UserCard.tsx` :

```

import { memo } from "react";
import type { GitHubUser } from "../types/github";

interface UserCardProps {
  user: GitHubUser;
}

// memo prevents re-render if props haven't changed
export const UserCard = memo(function UserCard({ user }: UserCardProps) {
  return (
    <a
      href={user.html_url}
      target="_blank"
      rel="noopener noreferrer"
      className="user-card"
      aria-label={`View ${user.login}'s GitHub profile`}
    >
      <img
        src={user.avatar_url}
        alt={`${user.login} avatar`}
        width={48}
        height={48}
        loading="lazy"
      />
      <div className="user-info">
        <span className="user-login">{user.login}</span>
        <span className="user-type">{user.type}</span>
      </div>
    </a>
  );
});

```

Create `/src/components/GitHubSearch.tsx` :

```

import { useMemo, useCallback, useId } from "react";
import { useGitHubSearch } from "../hooks/useGitHubSearch";
import { UserCard } from "./UserCard";

export function GitHubSearch() {
  const inputId = useId(); // Stable unique ID for accessibility
  const { query, setQuery, users, totalCount, status, errorMessage, clearResults } =
    useGitHubSearch();

  // useMemo: only recompute when users array reference changes
  const userList = useMemo(
    () => users.map((user) => <UserCard key={user.id} user={user} />),
    [users]
  );

  // useCallback: stable reference so SearchInput doesn't re-render unnecessarily

```

```

const handleChange = useCallback(
  (e: React.ChangeEvent<HTMLInputElement>) => {
    setQuery(e.target.value);
  },
  [setQuery]
);

return (
  <section className="github-search" aria-label="GitHub user search">
    <h1>GitHub User Search</h1>

    <div className="search-bar">
      <label htmlFor={inputId} className="sr-only">
        Search GitHub users
      </label>
      <input
        id={inputId}
        type="search"
        value={query}
        onChange={handleChange}
        placeholder="Type a username..."
        autoComplete="off"
        aria-busy={status === "loading"}
        aria-describedby="search-status"
      />
      {query && (
        <button onClick={clearResults} aria-label="Clear search">
          ×
        </button>
      )}
    </div>

    <div id="search-status" aria-live="polite" aria-atomic="true">
      {status === "loading" && <p className="status-loading">Searching...</p>}
      {status === "error" && (
        <p className="status-error" role="alert">
          {errorMessage}
        </p>
      )}
      {status === "success" && (
        <p className="status-count">
          {totalCount.toLocaleString()} users found
          {users.length < totalCount && ` (showing first ${users.length})`}
        </p>
      )}
    </div>

    {status === "success" && users.length === 0 && (
      <p className="no-results">No users found for "{query}"</p>
    )}

    <div className="user-grid" role="list">
      {userList}
    </div>
  </section>
);

```

```
    </div>
  </section>
);
}
```

Step 6: Wire It Up

In `/src/App.tsx` :

```
import { GitHubSearch } from "../components/GitHubSearch";
import "../App.css";

export default function App() {
  return (
    <main>
      <GitHubSearch />
    </main>
  );
}
```

Expected Output

When you run `npm run dev` and type "torvalds" into the search box:

1. After 450ms of no typing, a "Searching..." indicator appears.
 2. Results appear: Linus Torvalds' avatar, login, and a link to `github.com/torvalds`.
 3. The total count "1 users found" displays above the list.
 4. Clearing the input resets to the idle state with no API call made.
-

Common Pitfalls

Pitfall	Symptom	Fix
Missing AbortController cleanup	Stale results appear after fast typing	Add <code>return () => controller.abort()</code> in <code>useEffect</code>
No <code>[]</code> dependency array	Infinite fetch loop	Always specify deps; use <code>eslint-plugin-react-hooks</code>
Forgetting <code>memo</code> on <code>UserCard</code>	Every keystroke re-renders all cards	Wrap with <code>memo()</code> ; ensure key is stable (use <code>id</code> , not <code>index</code>)
GitHub API rate limit (60 req/hr unauthenticated)	403 errors	Add <code>Authorization: Bearer <token></code> header
<code>useId()</code> not used for label association	Accessibility failure	Always pair <code><label htmlFor></code> with <code>useId()</code>

System Design — Quiz: Test Your Knowledge

Type: Quiz | Duration: 20 min | XP: 50

Instructions

Select the best answer for each question. Read the explanation after each answer to deepen your understanding.

Question 1 — CAP Theorem

A distributed database guarantees that every read receives the most recent write or an error, and it continues operating even when network partitions occur — but it may not be

available to all nodes simultaneously. Which two CAP properties does this system prioritize?

- A) Availability and Partition Tolerance
- B) Consistency and Availability
- C) Consistency and Partition Tolerance ✓
- D) All three — CAP is always achievable

Explanation: The CAP theorem (Brewer, 2000) states that a distributed system can guarantee at most two of: Consistency (every read gets the latest write), Availability (every request gets a response), and Partition Tolerance (system operates despite network splits). Since network partitions are unavoidable in real distributed systems, you always face a C vs. A tradeoff. HBase and Zookeeper are CP systems; Cassandra and DynamoDB are AP systems; traditional RDBMS are CA (but assume no partitions).

Question 2 — Consistent Hashing

You have a caching cluster of 10 nodes. You use consistent hashing to route keys. You add 3 more nodes. Approximately what fraction of keys need to be remapped?

- A) 100% of keys move to the new nodes
- B) 3/13 of all keys — only those that now map to new nodes ✓
- C) 10/13 of all keys — all existing keys re-sort
- D) No keys move — consistent hashing is static

Explanation: With standard (mod N) hashing, adding a node remaps nearly all keys (since the modulus changes). Consistent hashing places both nodes and keys on a ring. When a node is added, only the keys in its arc on the ring move — roughly K/N keys, where K = total keys and N = new node count. Adding 3 nodes to 10 remaps approximately $3/13 \approx 23\%$ of keys. This is why consistent hashing is used in memcached, Cassandra, and distributed load balancers.

Question 3 — Database Sharding

Your `orders` table has 500 million rows. You decide to shard by `customer_id`. A business analyst needs to run: `SELECT * FROM orders WHERE order_date BETWEEN '2025-01-01' AND '2025-03-31'`. What problem will this query face?

- A) The query is invalid SQL when sharding is used

- B) The query must be sent to all shards and results merged (scatter-gather) ✓
- C) The query only runs on the shard containing `customer_id = 0`
- D) Sharding automatically indexes `order_date` across all shards

Explanation: Sharding by `customer_id` means rows for a customer live on a single shard — efficient for `WHERE customer_id = X` queries. But `order_date` is not the shard key, so the query engine has no way to know which shard(s) contain relevant dates. It must fan out to every shard (scatter), collect results, and merge them (gather). This is expensive and slow. Common mitigations: use a date-based secondary shard key, maintain a separate analytics replica, or use a data warehouse (Redshift, BigQuery) for analytical queries.

Question 4 — Load Balancing Algorithms

You have a cluster of application servers. Some requests process in 1ms (static assets) and others take 500ms (complex computations). Your servers have varying CPU loads. Which load balancing algorithm best handles this mixed workload?

- A) Round Robin — distributes evenly by count
- B) Random — statistically even over time
- C) Least Connections ✓
- D) IP Hash — sticky sessions per client

Explanation: Round Robin and Random treat all requests equally, so slow requests can pile up on already-busy servers. IP Hash creates sticky sessions (useful for stateful apps) but ignores server load. Least Connections routes each new request to the server with the fewest active connections — naturally directing traffic away from servers processing the slow 500ms requests. For CPU-intensive workloads, Weighted Least Connections (accounting for server CPU/RAM capacity) is even better.

Question 5 — Caching Strategies

Your e-commerce site writes order records to a database. You add a cache in front. You choose **write-through** caching. Which statement accurately describes this strategy's trade-off?

- A) Writes go to cache only; the database is updated lazily — fast writes, risk of data loss ✓ (this describes write-back)
- B) Writes update both cache and database synchronously — consistent but higher write latency ✓

- C) Writes bypass the cache entirely and go directly to the database — reads may be stale
- D) Writes are batched and flushed to the database every 60 seconds

Explanation: Write-through: write hits cache AND database at the same time. Every cached entry is consistent with the DB. Trade-off: write latency doubles (two round trips). Write-back (write-behind): write to cache only, database updated asynchronously — fast writes but risk losing data if cache fails. Write-around: writes go directly to DB, bypassing cache — cache only fills on reads (cache-aside). Good for write-heavy data that won't be re-read soon.

Question 6 — Message Queues

You are designing a system where an order service publishes events and three downstream services (inventory, billing, email) must each process every order event independently. Which messaging pattern fits best?

- A) Point-to-point queue — one consumer per message
- B) Publish-Subscribe (Pub/Sub) with topics/fanout exchange ✓
- C) Request-Reply — synchronous RPC over the queue
- D) Competing consumers — multiple workers share a single queue

Explanation: Point-to-point (standard queue) delivers each message to exactly one consumer — only one of your three services would receive each order. Competing consumers is a point-to-point pattern for scaling a single logical consumer. Request-Reply is synchronous. Pub/Sub (SNS fanout → multiple SQS queues, or Kafka topics with multiple consumer groups) delivers each message to all subscribers independently. This is the correct pattern: one topic, three subscriptions, each service receives every event.

Question 7 — CDN (Content Delivery Network)

A user in Tokyo requests an image from your origin server in Virginia. The CDN PoP (Point of Presence) in Tokyo does not have the image cached (cache MISS). What happens next, and what is the primary latency benefit after the first request?

- A) The CDN rejects the request; the user's browser falls back to the origin directly
- B) The Tokyo PoP fetches from the origin, caches the response, and serves future Tokyo users from the PoP ✓
- C) The CDN returns a 404 until the image is manually pushed to the Tokyo PoP

- D) The user is redirected to the nearest region that has the content cached

Explanation: On a cache miss, the CDN PoP acts as a proxy: it fetches from the origin (Virginia), stores the response according to cache-control headers, and returns it to the user. All subsequent Tokyo users get the image from the Tokyo PoP (~5-10ms) instead of the Virginia origin (~180ms round-trip), dramatically reducing latency. CDNs also reduce origin load, improve resilience, and can absorb DDoS traffic. Cache invalidation (ensuring stale content is purged) is the hard part — use versioned asset filenames (`app.a3f9d.js`) to bypass it entirely.

Question 8 — Database Indexing

A table `events` has 50 million rows. The query `SELECT * FROM events WHERE user_id = 42 AND event_type = 'purchase' ORDER BY created_at DESC` runs in 12 seconds. You add a composite index. Which index definition will most effectively optimise this specific query?

- A) `INDEX (user_id)`
- B) `INDEX (event_type, user_id, created_at)`
- C) `INDEX (user_id, event_type, created_at)` ✓
- D) `INDEX (created_at, user_id, event_type)`

Explanation: Composite index column order matters. The rule: equality predicates first, range/sort columns last. Here: `user_id = 42` (equality), `event_type = 'purchase'` (equality), `ORDER BY created_at DESC` (sort). Index `(user_id, event_type, created_at)` allows the DB to seek directly to the matching rows AND return them pre-sorted by `created_at`, eliminating both an index scan and a filesort. Option B places `event_type` first — if a query only provides `user_id`, this index won't be used (leading column mismatch). Option D starts with `created_at`, which is a range/sort column — useless for filtering by user.

AWS Solutions Architect — Serverless Image Processing Pipeline

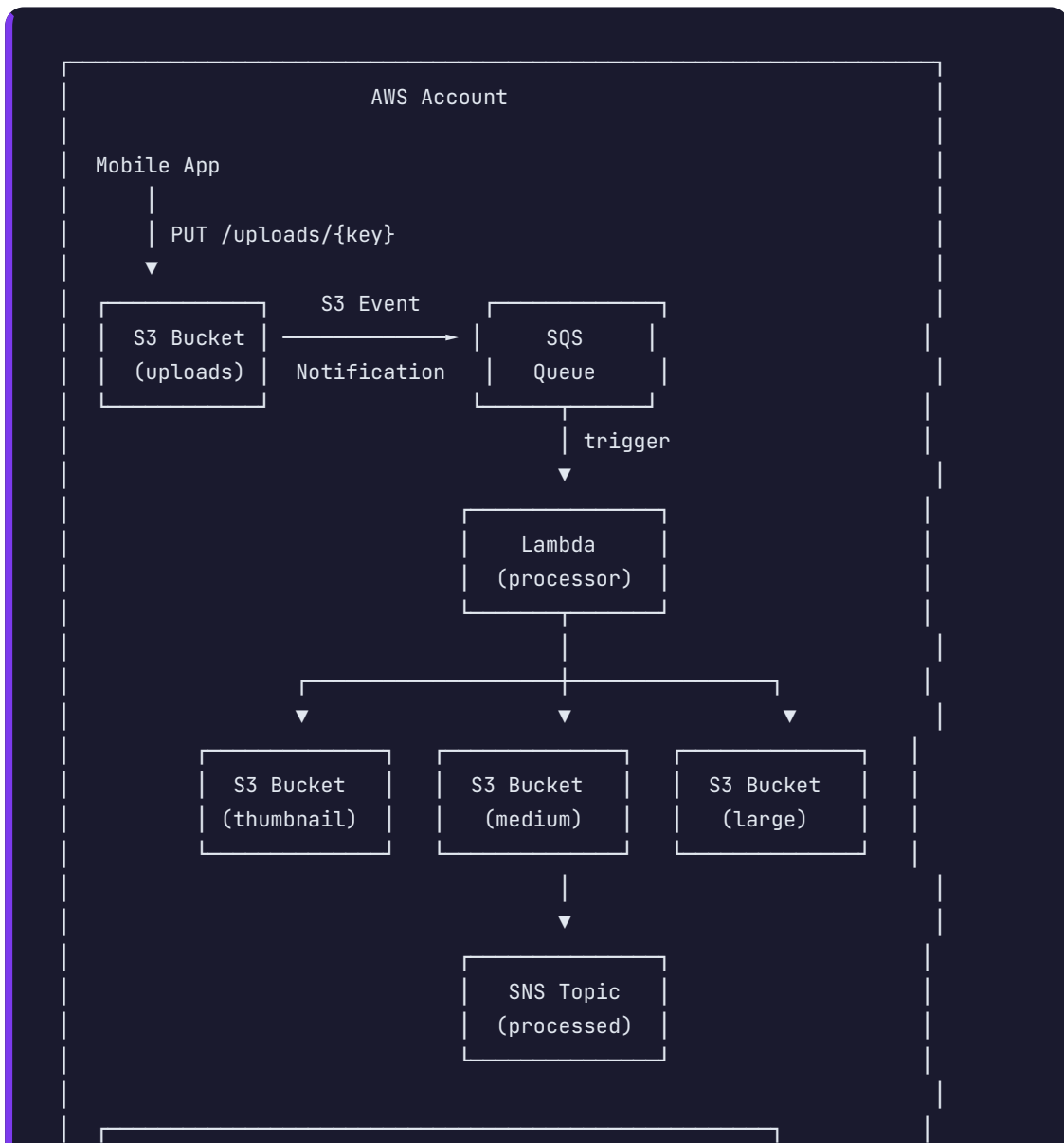
Type: Project | Duration: 60 min | XP: 150

Project Brief

You are the lead architect at a media startup. Users upload photos through a mobile app. The system must automatically resize uploaded images to three standard resolutions (thumbnail 150×150, medium 800×600, large 1920×1080), strip EXIF metadata for privacy, store the processed images, and notify downstream services when processing completes — all without managing any servers.

Learning Goals: S3 event notifications, Lambda function design, SQS for reliable decoupling, CloudWatch for observability, IAM least-privilege roles, and CDK for infrastructure-as-code.

Architecture Overview



```
CloudWatch: Lambda metrics, custom metrics,  
SQS dead-letter queue alarm, X-Ray tracing
```

Implementation

Phase 1: Bootstrap the CDK Project

```
# Prerequisites  
npm install -g aws-cdk  
cdk --version # Should be 2.x  
  
# Bootstrap  
mkdir image-pipeline && cd image-pipeline  
cdk init app --language typescript  
npm install @aws-cdk/aws-lambda-nodejs sharp @aws-sdk/client-s3 @aws-sdk/client-sns  
npm install --save-dev @types/aws-lambda esbuild
```

Phase 2: Create S3 Buckets and SQS Queue

In `/lib/image-pipeline-stack.ts` :

```
import * as cdk from "aws-cdk-lib";  
import * as s3 from "aws-cdk-lib/aws-s3";  
import * as sqs from "aws-cdk-lib/aws-sqs";  
import * as sns from "aws-cdk-lib/aws-sns";  
import * as lambda from "aws-cdk-lib/aws-lambda-nodejs";  
import * as iam from "aws-cdk-lib/aws-iam";  
import * as cloudwatch from "aws-cdk-lib/aws-cloudwatch";  
import * as s3n from "aws-cdk-lib/aws-s3-notifications";  
import * as lambdaEventSources from "aws-cdk-lib/aws-lambda-event-sources";  
import { Construct } from "constructs";  
import { Duration, RemovalPolicy } from "aws-cdk-lib";  
  
export class ImagePipelineStack extends cdk.Stack {  
  constructor(scope: Construct, id: string, props?: cdk.StackProps) {  
    super(scope, id, props);  
  
    // --- Storage ---  
  
    const uploadsBucket = new s3.Bucket(this, "UploadsBucket", {  
      bucketName: `image-uploads-${this.account}-${this.region}`,
```

```

    removalPolicy: RemovalPolicy.DESTROY,
    autoDeleteObjects: true,
    lifecycleRules: [
      // Delete original uploads after 30 days (already processed)
      { expiration: Duration.days(30), prefix: "uploads/" },
    ],
    cors: [
      {
        allowedMethods: [s3.HttpMethods.PUT],
        allowedOrigins: ["*"], // Restrict to your app domain in production
        allowedHeaders: ["*"],
      },
    ],
  });

const processedBucket = new s3.Bucket(this, "ProcessedBucket", {
  bucketName: `image-processed-${this.account}-${this.region}`,
  removalPolicy: RemovalPolicy.RETAIN,
  publicReadAccess: false,
  // Enable CloudFront later for CDN delivery
});

// --- Dead Letter Queue ---

const dlq = new sqs.Queue(this, "ProcessingDLQ", {
  queueName: "image-processing-dlq",
  retentionPeriod: Duration.days(14),
});

// --- Main Processing Queue ---

const processingQueue = new sqs.Queue(this, "ProcessingQueue", {
  queueName: "image-processing-queue",
  visibilityTimeout: Duration.seconds(300), // Must be ≥ Lambda timeout
  deadLetterQueue: {
    queue: dlq,
    maxReceiveCount: 3, // After 3 failures, move to DLQ
  },
});

// --- SNS Topic for Completion Events ---

const completionTopic = new sns.Topic(this, "CompletionTopic", {
  topicName: "image-processing-complete",
});

// --- Wire S3 → SQS Notification ---

uploadsBucket.addEventNotification(
  s3.EventType.OBJECT_CREATED,
  new s3n.SqsDestination(processingQueue),
  { prefix: "uploads/", suffix: ".jpg" }
);

```

```

uploadsBucket.addEventNotification(
    s3.EventType.OBJECT_CREATED,
    new s3n.SqsDestination(processingQueue),
    { prefix: "uploads/", suffix: ".png" }
);

// --- Lambda IAM Role (Least Privilege) ---

const processorRole = new iam.Role(this, "ProcessorRole", {
    assumedBy: new iam.ServicePrincipal("lambda.amazonaws.com"),
    managedPolicies: [
        iam.ManagedPolicy.fromAwsManagedPolicyName(
            "service-role/AWSLambdaBasicExecutionRole"
        ),
    ],
});

uploadsBucket.grantRead(processorRole);
processedBucket.grantWrite(processorRole);
completionTopic.grantPublish(processorRole);

// --- Lambda Function ---

const processorFn = new lambda.NodejsFunction(this, "ImageProcessor", {
    entry: "lambda/processor.ts",
    handler: "handler",
    runtime: cdk.aws_lambda.Runtime.NODEJS_20_X,
    role: processorRole,
    timeout: Duration.seconds(120),
    memorySize: 1024, // Sharp is memory-intensive; more RAM = more CPU
    environment: {
        PROCESSED_BUCKET: processedBucket.bucketName,
        COMPLETION_TOPIC_ARN: completionTopic.topicArn,
        NODE_OPTIONS: "--enable-source-maps",
    },
    bundling: {
        nodeModules: ["sharp"], // Bundle as native module for Lambda
        forceDockerBundling: true, // Build in Docker for correct Lambda architecture
    },
});

// --- Trigger Lambda from SQS ---

processorFn.addEventSource(
    new lambdaEventSources.SqsEventSource(processingQueue, {
        batchSize: 5, // Process up to 5 images per invocation
        reportBatchItemFailures: true, // Partial batch success support
    })
);

// --- CloudWatch Alarms ---

new cloudwatch.Alarm(this, "DLQAlarm", {

```

```

        metric: dlq.metricApproximateNumberOfMessagesVisible(),
        threshold: 1,
        evaluationPeriods: 1,
        alarmDescription: "Messages in DLQ – image processing failures detected",
    });

    new cloudwatch.Alarm(this, "ProcessingErrorAlarm", {
        metric: processorFn.metricErrors({ period: Duration.minutes(5) }),
        threshold: 5,
        evaluationPeriods: 2,
        alarmDescription: "Lambda processor error rate too high",
    });
}
}

```

Phase 3: Lambda Processor Implementation

Create `/lambda/processor.ts` :

```

import {
    SQSHandler,
    SQSRecord,
    SQSBatchResponse,
    SQSBatchItemFailure,
} from "aws-lambda";
import { S3Client, GetObjectCommand, PutObjectCommand } from "@aws-sdk/client-s3";
import { SNSClient, PublishCommand } from "@aws-sdk/client-sns";
import sharp from "sharp";
import { Readable } from "stream";

const s3 = new S3Client({});
const sns = new SNSClient({});

const PROCESSED_BUCKET = process.env.PROCESSED_BUCKET!;
const COMPLETION_TOPIC_ARN = process.env.COMPLETION_TOPIC_ARN!;

const SIZES = [
    { label: "thumbnail", width: 150, height: 150, fit: "cover" as const },
    { label: "medium", width: 800, height: 600, fit: "inside" as const },
    { label: "large", width: 1920, height: 1080, fit: "inside" as const },
];

async function streamToBuffer(stream: Readable): Promise<Buffer> {
    const chunks: Buffer[] = [];
    for await (const chunk of stream) {
        chunks.push(Buffer.isBuffer(chunk) ? chunk : Buffer.from(chunk));
    }
    return Buffer.concat(chunks);
}

```

```

async function processImage(record: SQSRecord): Promise<void> {
    const body = JSON.parse(record.body);
    const s3Event = body.Records?.[0];
    if (!s3Event) throw new Error("Invalid SQS record: missing S3 event");

    const sourceBucket = s3Event.s3.bucket.name;
    const sourceKey = decodeURIComponent(s3Event.s3.object.key.replace(/\+/g, " "));

    console.log(`Processing: s3://${sourceBucket}/${sourceKey}`);

    // Fetch original image
    const { Body } = await s3.send(
        new GetObjectCommand({ Bucket: sourceBucket, Key: sourceKey })
    );
    const imageBuffer = await streamToBuffer(Body as Readable);

    // Strip EXIF metadata and get image metadata
    const image = sharp(imageBuffer).withMetadata(false); // false = strip all metadata
    const metadata = await image.metadata();

    const baseKey = sourceKey.replace(/^uploads\/, "").replace(/\.([^.]+"$/ , "");
    const format = (metadata.format as "jpeg" | "png") ?? "jpeg";

    // Process all sizes in parallel
    const uploads = SIZES.map(async ({ label, width, height, fit }) => {
        const processed = await sharp(imageBuffer)
            .withMetadata(false)
            .resize({ width, height, fit, withoutEnlargement: true })
            .toFormat(format, { quality: 85 })
            .toBuffer();

        const destKey = `${label}/${baseKey}.${format}`;
        await s3.send(
            new PutObjectCommand({
                Bucket: PROCESSED_BUCKET,
                Key: destKey,
                Body: processed,
                ContentType: `image/${format}`,
                CacheControl: "public, max-age=31536000, immutable",
                Metadata: {
                    "original-key": sourceKey,
                    "processed-at": new Date().toISOString(),
                },
            })
        );
        return destKey;
    });

    const processedKeys = await Promise.all(uploads);

    // Publish completion event
    await sns.send(
        new PublishCommand({

```

```

    TopicArn: COMPLETION_TOPIC_ARN,
    Message: JSON.stringify({
      event: "IMAGE_PROCESSED",
      sourceKey,
      processedAt: new Date().toISOString(),
      outputs: processedKeys,
    }),
    MessageAttributes: {
      event: { DataType: "String", StringValue: "IMAGE_PROCESSED" },
    },
  })
}
);

console.log(`Completed processing. Outputs: ${processedKeys.join(", ")}`);
}

export const handler: SQSHandler = async (event): Promise<SQSBatchResponse> => {
  const failures: SQSBatchItemFailure[] = [];

  await Promise.allSettled(
    event.Records.map(async (record) => {
      try {
        await processImage(record);
      } catch (err) {
        console.error(`Failed to process record ${record.messageId}:`, err);
        // Report this item as failed; SQS will retry or move to DLQ
        failures.push({ itemIdentifier: record.messageId });
      }
    })
  );

  return { batchItemFailures: failures };
};

```

Phase 4: Useful AWS CLI Commands

```

# Deploy the stack
cdk deploy ImagePipelineStack

# Test: upload a sample image
aws s3 cp ./test-photo.jpg \
  s3://image-uploads-$(aws sts get-caller-identity --query Account --output text)-us-east-1/uploads

# Watch Lambda logs in real time
aws logs tail /aws/lambda/ImagePipelineStack-ImageProcessor \
  --follow --format short

# Check SQS queue depth
aws sqs get-queue-attributes \
  --queue-url $(aws sqs get-queue-url --queue-name image-processing-queue --query QueueUrl --output

```

```

--attribute-names ApproximateNumberOfMessages,ApproximateNumberOfMessagesNotVisible

# Check Dead Letter Queue (failures)
aws sqs get-queue-attributes \
  --queue-url $(aws sqs get-queue-url --queue-name image-processing-dlq --query QueueUrl --output text) \
  --attribute-names ApproximateNumberOfMessages

# List processed outputs
aws s3 ls s3://image-processed-$(aws sts get-caller-identity --query Account --output text)-us-east-1

# Invoke Lambda manually for debugging (bypasses SQS)
aws lambda invoke \
  --function-name ImagePipelineStack-ImageProcessor \
  --payload file://test-event.json \
  --log-type Tail \
  response.json && cat response.json

```

⚠ Warning: The Lambda uses `sharp` which includes native binaries. Always set `forceDockerBundling: true` in CDK to build the native module for the Lambda execution environment (`linux/x86_64` or `linux/arm64`). Building on macOS without Docker will produce binaries that crash on Lambda with `Error: 'linux-x64-gnu' binding not found .`

Evaluation Criteria

Criterion	Points	What's Assessed
Infrastructure deploys without errors	20	<code>cdk deploy</code> succeeds; all resources created
Image upload triggers Lambda via SQS	20	S3 event → SQS → Lambda invocation confirmed in CloudWatch
All 3 output sizes present in S3	25	<code>thumbnail/</code> , <code>medium/</code> , <code>large/</code> prefixes exist with correct keys
EXIF metadata stripped	10	<code>exiftool</code> on output image shows no GPS/author data
Partial batch failure handling	10	One bad record in batch doesn't block other records

Criterion	Points	What's Assessed
DLQ alarm triggers on failure	10	Force a failure; confirm CloudWatch alarm enters ALARM state
IAM least privilege verified	5	Lambda role has no <code>s3:*</code> — only <code>s3:GetObject</code> and <code>s3:PutObject</code>

Total: 100 points

Pro Tip: For production, add an S3 Pre-signed URL endpoint (API Gateway + Lambda) so your mobile app uploads directly to S3 without routing image bytes through your backend. This reduces server load, lowers costs, and leverages S3's multi-part upload for large files. The pre-signed URL grants a temporary, scoped permission for exactly one `PutObject` operation.

Extension Challenges

1. **Add WebP conversion** — Modify the processor to also output `.webp` format alongside the original format. WebP is ~30% smaller than JPEG at equivalent quality.
2. **Content moderation gate** — Before resizing, call Amazon Rekognition `DetectModerationLabels`. If confidence > 90% on an unsafe category, move the file to a quarantine bucket instead of processing it.
3. **CloudFront CDN** — Add a CloudFront distribution in front of the processed bucket. Use signed URLs or signed cookies for private content access control.
4. **Cost optimisation** — Add an S3 Intelligent-Tiering lifecycle rule on the processed bucket. After 90 days, infrequently accessed images move to cheaper storage automatically.

End of sample lesson content — 5 lessons covering Java, Python, React 19, System Design, and AWS.